Initiation à PowerShell

Module 05 – Réalisation d'un script

Objectifs

- Mettre en place un script fonctionnel

Réalisation d'un script

Comment structurer un script

- Avant d'entreprendre la rédaction d'un script, il est nécessaire de :
  - Déterminer son utilité
  - Décrire le mode de fonctionnement attendu
  - Formaliser un algorithme de son fonctionnement
- L'algorithmique est la clé de voûte du Scripting. La diapositive suivante illustre l'approche algorithmique de la préparation des pâtes.

Réalisation d'un script

Comment structurer un script

Algorithme de préparation des pâtes :

1. Remplir
2. La n
3. Mettre les pâtes dans
4. Les laisser cuire 10 à 15 min
5. Les égoutter
6. En fonction du t
7. Définir la
8. Les égou

L'eau ne bout pas, le temps de c… son est fa…

Elles risquent d'être fades…

Durée exacte en fonction du type de pâtes ?



Réalisation d'un script

Comment structurer un script

- Il est très rare de réaliser dès le premier essai et la première réflexion le script parfait (surtout sur des scripts un peu avancés…).
- C'est la phase de tests qui nous permettra d'avancer dans notre réflexion et d'améliorer l'algorithme initial.
- Il faut donc y aller étape par étape et tester ligne par ligne notre script.
- Et surtout, commentez votre script !

Réalisation d'un script

Comment structurer un script

- Il est généralement pertinent d'écrire sur papier la structure de notre script :
  - Actions à réaliser
  - Tests à effectuer
  - Vérifications
  - Boucles
- Le tout dans une logique chronologique

Préparer son script avant de l'écrire, c'est tout sauf une perte de temps !

Réalisation d'un script

Comment structurer un script : le départ

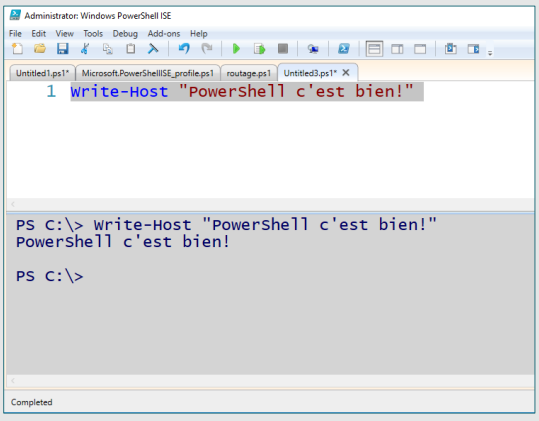
- La réalisation d'un script peut se faire à l'aide d'un IDE compatible avec PowerShell.
- Un IDE est un logiciel dédié à l'écriture de code informatique en y ajoutant des fonctions de correction ou de vérification du code.
- ISE est un logiciel qui propose un environnement d'écriture des commandes PowerShell, et également une console permettant de tester le script en temps réel.

Réalisation d'un script

PowerShell ISE

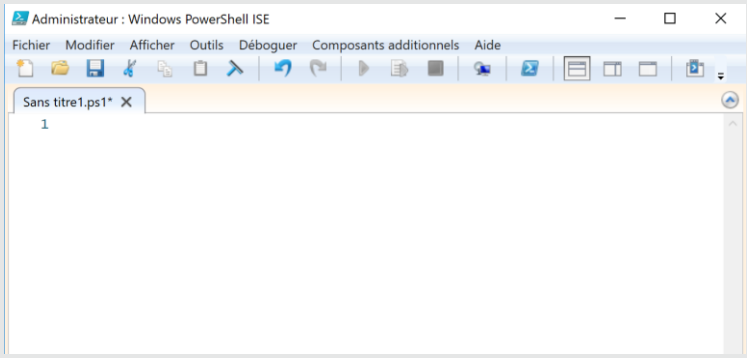
ISE : Integrated Scripting Environment

- C'est une application adaptée pour travailler avec PowerShell
- Présent par défaut depuis PowerShell version 3
- Présente de nombreuses fonctionnalités orientées Scripting



Réalisation d'un script

PowerShell ISE : fonctionnalités



IntelliSense : complétion graphique

Permet de rechercher et utiliser des paramètres plus rapidement

Coloration syntaxique

Snippets : syntaxe des structures de scripting

Détection des erreurs en temps réel

Démonstration

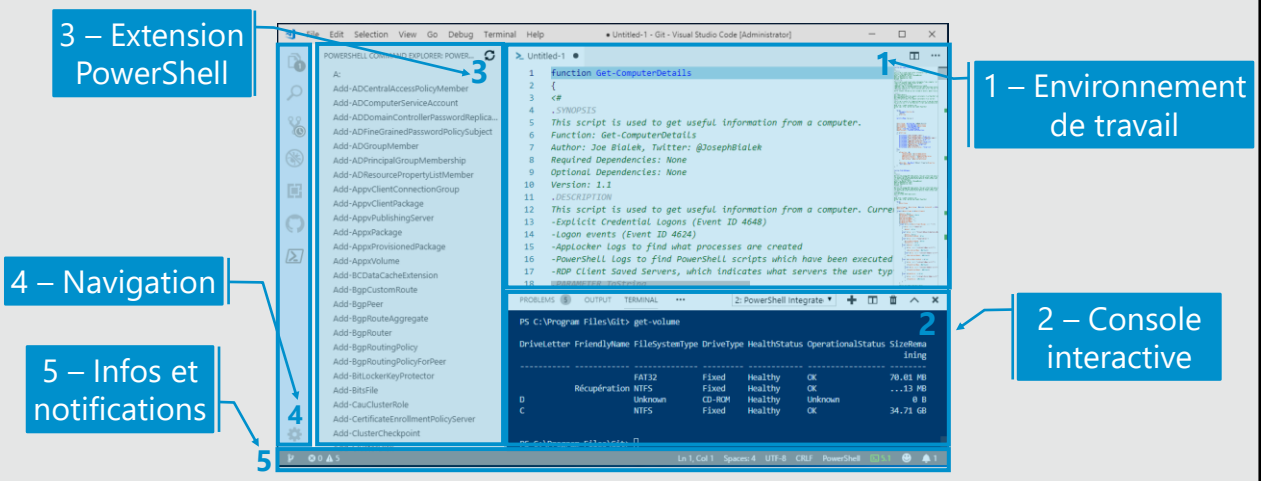
Réalisation d'un script

Visual Studio Code (VS Code)

- Windows PowerShell ISE n'est disponible que sur les systèmes Windows.
- VS Code : interface de développement open source
  - Disponible notamment pour Linux, MacOS et Windows
  - Légère et fluide
  - Multilingue (Shell, Python, Perl…)
  - Possibilité d'ajout d'extensions (nouveaux langages, fonctionnalités, GitHub…)
  - Débogueur
  - Outil de recherche
  - …

Réalisation d'un script

Virtual Studio Code (VSCode)



Démonstration

Réalisation d'un script

Les commentaires

1 – Commentaire simple

3 – Commentaire après une commande

5 – 7 Il est possible de commenter une structure ou un ensemble de lignes en utilisant les chevrons et le dièze.

```
1  # Bonjour ceci est un commentaire.
2
3  get-command # Ceci est un commentaire.
4
5  <#if ($x -eq "TOTO"){
6  Write-Host "Bonjour Toto"
7  }#> #Cette structure est commenté
```

Réalisation d'un script

Les enchaînements de commande

Lors de la réalisation d'un script, il est utile de pouvoir enchaîner plusieurs commandes sur une seule ligne du script.

- À la différence du Pipe, chaque commande ne sera pas dépendante du résultat de la commande précédente.
- On utilise le caractère ; pour exécuter deux commandes sur une ligne de script.

```
1   Get-LocalUser -Name "Toto"    ; Get-LocalGroup -Name "Administrateur"
```

- Il est également possible de répartir les paramètres d'une commande sur plusieurs lignes.

```
1  do {
2      $x=Read-Host "Veuillez saisir votre valeur"
3      if ($x -gt 10){
4
5          Write-Host "Bonjour et bienvenue la valeur est trop haute " `
6                -ForegroundColor red
7                    }
8  } while ($x-lt 10)
```

Le « Backtick » ou « Anti-Quotte » s'obtient avec « Alt Gr », « 7 » puis « espace ».

Démonstration

Réalisation d'un script

La gestion des erreurs

- Toutes les erreurs générées dans le Shell courant sont stockées dans la variable de type tableau `$Error`.
- Pour obtenir la dernière erreur survenue `$Error[0]` .

Il est possible pour une commande ciblée, à son lancement, de redéfinir le **nom de la variable qui consignera ses erreurs**

```
PS > Get-ChildItem C:\Dossier -ErrorVariable Erreurs
```


Réalisation d'un script

La gestion des erreurs

- Quand une erreur survient lors de l'exécution d'une commande, deux comportements sont possibles :
  - La commande s'interrompt (erreur bloquante)
  - L'erreur est remontée au Shell mais la commande poursuit son exécution (erreur non bloquante)
- Le comportement du Shell en cas d'erreurs non bloquantes peut être géré :
  - Pour **la commande**, via le paramètre `-ErrorAction`
  - Pour **l'environnement** d'exécution courant, via la variable `$ErrorActionPreference`

Réalisation d'un script

La gestion des erreurs

Valeurs pouvant être affectées à la variable `$ErrorActionPreference` et au paramètre `-ErrorAction`

Valeur	Comportement résultant
Continue	Affiche l'erreur mais continue (niveau par défaut)
Ignore	Ignore l'erreur (celle-ci n'est pas stockée dans la variable d'erreur)
Inquire	Interaction avec l'utilisateur pour savoir l'action a réaliser
SilentlyContinue	Ignore l'erreur mais la consigne dans la variable d'erreur
Stop	Arrêt de la commande
Suspend	Suspend le workflow en cours

Réalisation d'un script

La gestion des erreurs : Try Catch (Finally…)

Couramment appelée Try Catch, la structure `Try Catch Finally` permet de gérer efficacement les erreurs.

```
1   try { Get-ChildItem c:\test -ErrorAction Stop}
2   catch {write-host "le dossier n'éxiste pas"  }
3   finally {write-host "fin du test"}
```

- Si le dossier existe, son contenu sera affiché.
- Si le dossier n'existe pas, le message contenu dans la commande `catch` s'affichera.
- Dans tous les cas, le message contenu dans la commande `finally` s'affichera.

L'avantage est qu'en cas de dossier non présent,
le script continue son exécution sans arrêter la totalité du script.

Démonstration

Réalisation d'un script

Les fonctions : présentation

- Les fonctions sont des structures très utiles à la réalisation de scripts.
- Elles permettent :
  - D'éviter la réécriture de code
  - D'alléger et de clarifier la lecture du code
  - De simplifier la maintenance
- Créer une fonction revient à créer une nouvelle commande.

Réalisation d'un script

Les fonctions : utilisation

- Une **fonction** doit être **déclarée** avant de pouvoir être **appelée**.
- La déclaration de fonctions est généralement faite en début de script.

```
PS > Function NomDeLaFonction
          {
               Code à intégrer dans la fonction
          }
```

Donner des noms explicites à vos fonctions !

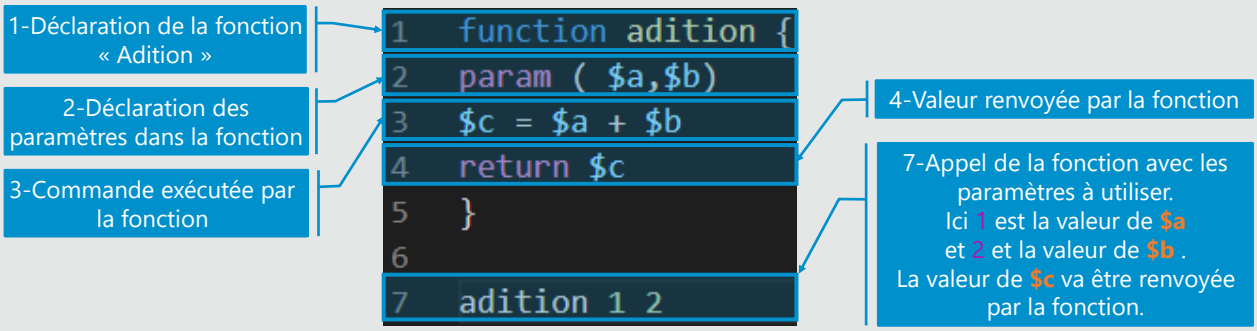
- Une fonction est appelée comme une commande.

```
PS > NomDeLaFonction
```


Réalisation d'un script

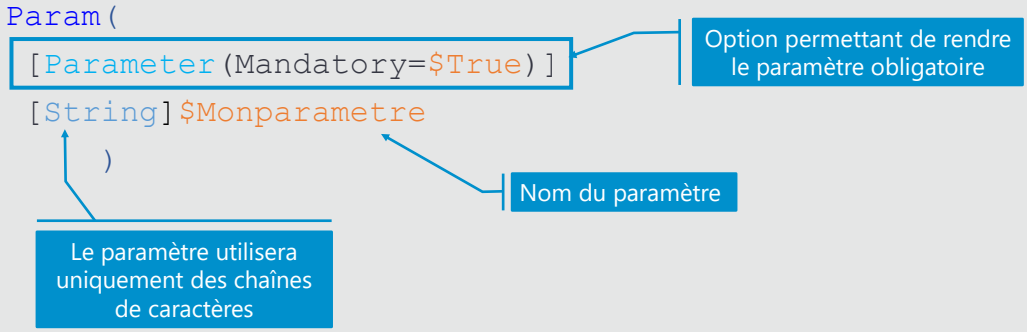
Les fonctions : les paramètres

- Les fonctions acceptent des paramètres comme n'importe quelle commande.
- Les paramètres de fonction sont à déclarer en même temps que la fonction.



Réalisation d'un script

Les fonctions : les paramètres avancés




```
PS > ./Monscript.ps1 -Monparametre toto
PS > Ma-Fonction -Monparametre toto
```


Démonstration

Réalisation d'un script

Le Remoting

- La fonctionnalité PowerShell Remoting permet :
  - D'accéder au Shell d'une machine distante
  - D'exécuter des commandes sur des machines distantes
- Elle s'appuie sur le protocole WS-MAN pour son fonctionnement.
- Le service WinRM est utilisé.

Réalisation d'un script

Le Remoting

- Pour pouvoir utiliser le Remoting PowerShell, cette fonctionnalité doit être active sur la machine cible (la machine distante)
  - C'est le cas (par défaut) sur les OS Serveurs depuis Windows 2012
  - Cette fonctionnalité est désactivée (par défaut) sur les postes clients
- La fonctionnalité **Remoting** est activable
  - Au moyen de la commande `Enable-PSRemoting`
  - Par stratégie de groupe

Démonstration

Réalisation d'un script

PSSession : ouverture de session à distance

- Il est possible d'ouvrir un Shell à distance (similaire à SSH).

```
PS > Enter-PSSession -Computername DC01
PS  [DC01] >
```

- Le prompt change, on peut y voir le nom du serveur distant.
- Pour sortir de la session :

```
PS > Exit-PSSession
```

À l'issue de la commande de sortie, la session existe toujours.

Réalisation d'un script

PSSession

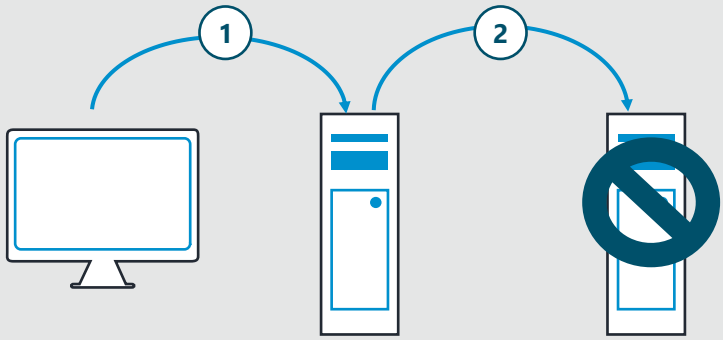
- On peut stocker une PSSession dans une variable, afin de la rappeler plus facilement par la suite.
- Une PSSession peut être persistante.

```
PS > $Session = New-PSSession -Computername DC01
PS > Disconnect-PSSession -Session $Session
PS > Get-PSSession -Computername DC01
PS > Get-PSSession -Computername DC01 | Connect-PSSession
PS > $Session
PS > Get-PSSession | Remove-PSSession
```


Réalisation d'un script

MultiHop

Il n'est, par défaut, pas possible d'utiliser une machine pour faire un « rebond » sur une autre avec les PSSessions.



Réalisation d'un script

PSSession et import de modules

On peut importer des modules d'une session distante pour utiliser ses commandes dans la session locale. Cela permet de limiter l'installation de modules sur le parc.

```
PS > $Session = Enter-PSSession -Computername DC01
PS > Get-Module -PSSession $Session -ListAvailable
PS > Import-Module -PSSession $Session -name ActiveDirectory -Prefix DC
PS > Help Get-DCADUser
PS > Get-DCADUser -Filter *
```


Réalisation d'un script

Invoke-Command

- Commande permettant de demander l'exécution d'une commande à plusieurs machines en même temps :

```
PS > Invoke-Command -Computername DC01,SRV01 -ScriptBlock {
Get-Process PowerShell* }
```

- On peut également passer des variables locales dans une session à distance en utilisant le `$Using:Variable`

```
$Process = ''Powershell*''
PS > Invoke-Command -Computername DC01,SRV01 -ScriptBlock {
Get-Process $Using:Process}
```

Démonstration

Conclusion

- Vous savez mettre en place un script fonctionnel